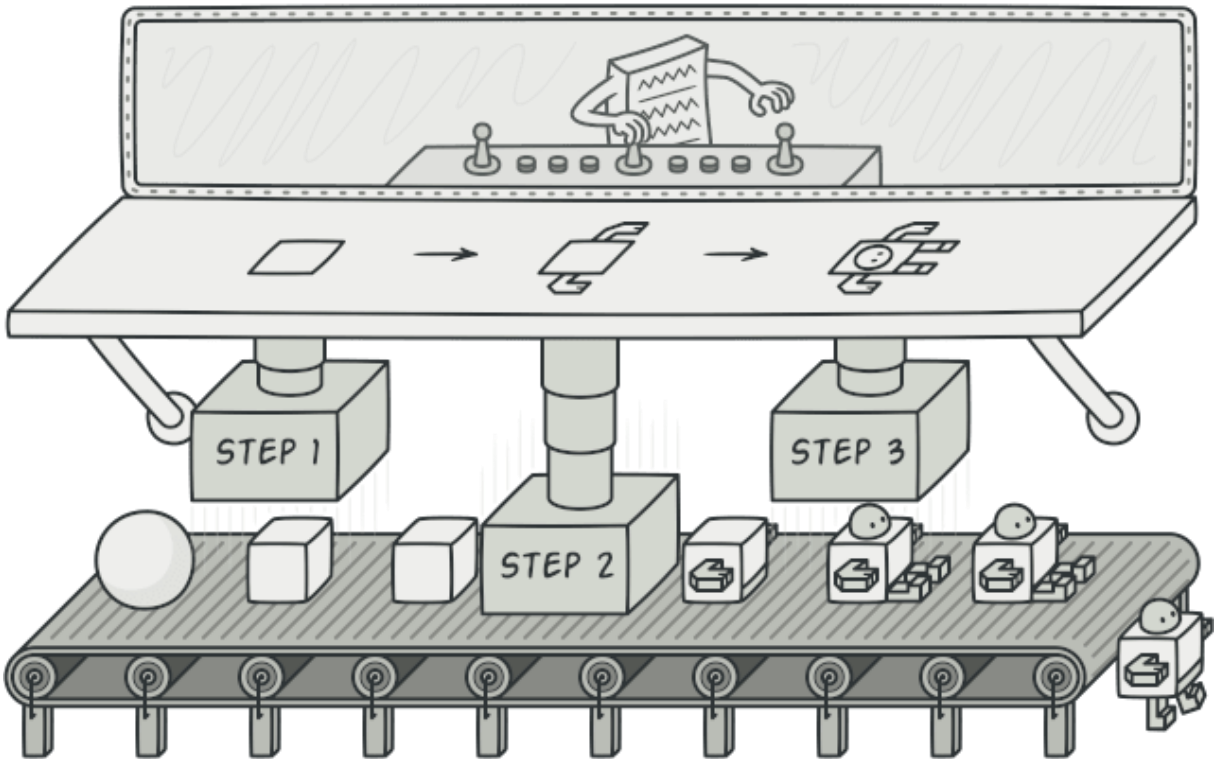
# Builder

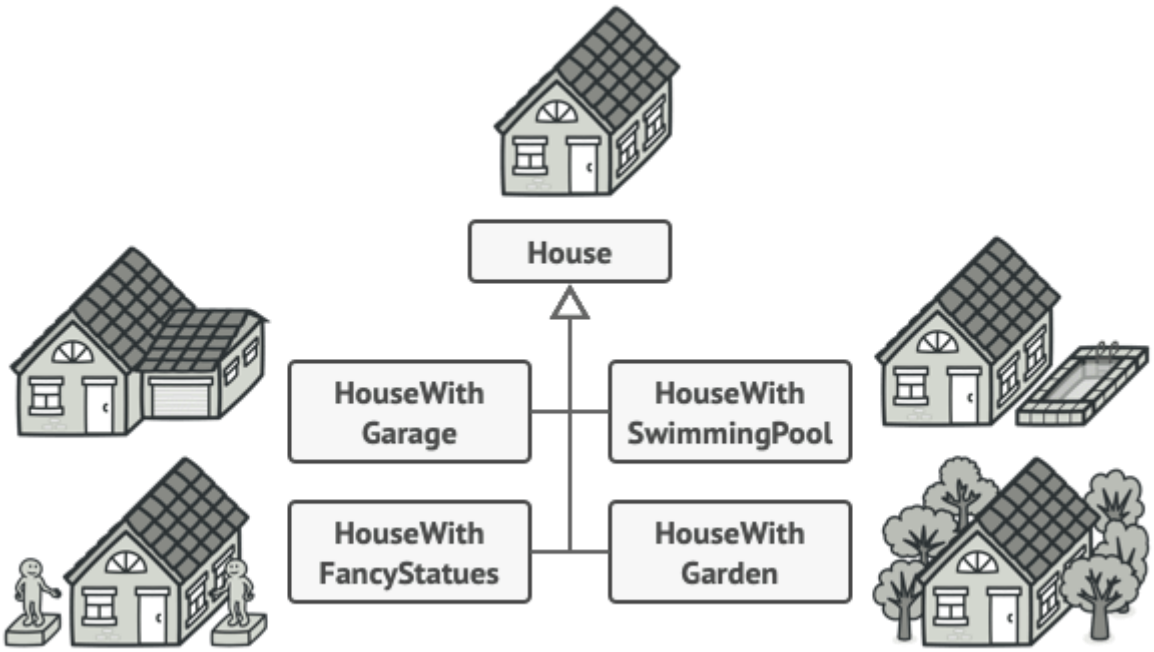
## Absicht

**Builder** ist ein kreatives Entwurfsmuster, mit dem Sie komplexe Objekte schrittweise konstruieren können. Mit dem Muster können Sie mit demselben Konstruktionscode verschiedene Typen und Darstellungen eines Objekts erstellen.




## Problem

Stellen Sie sich ein komplexes Objekt vor, das eine mühsame, schrittweise Initialisierung vieler Felder und verschachtelter Objekte erfordert. Dieser Initialisierungscode ist normalerweise in einem riesigen Konstruktor mit unzähligen Parametern verborgen. Oder noch schlimmer: über den gesamten Client-Code verstreut.

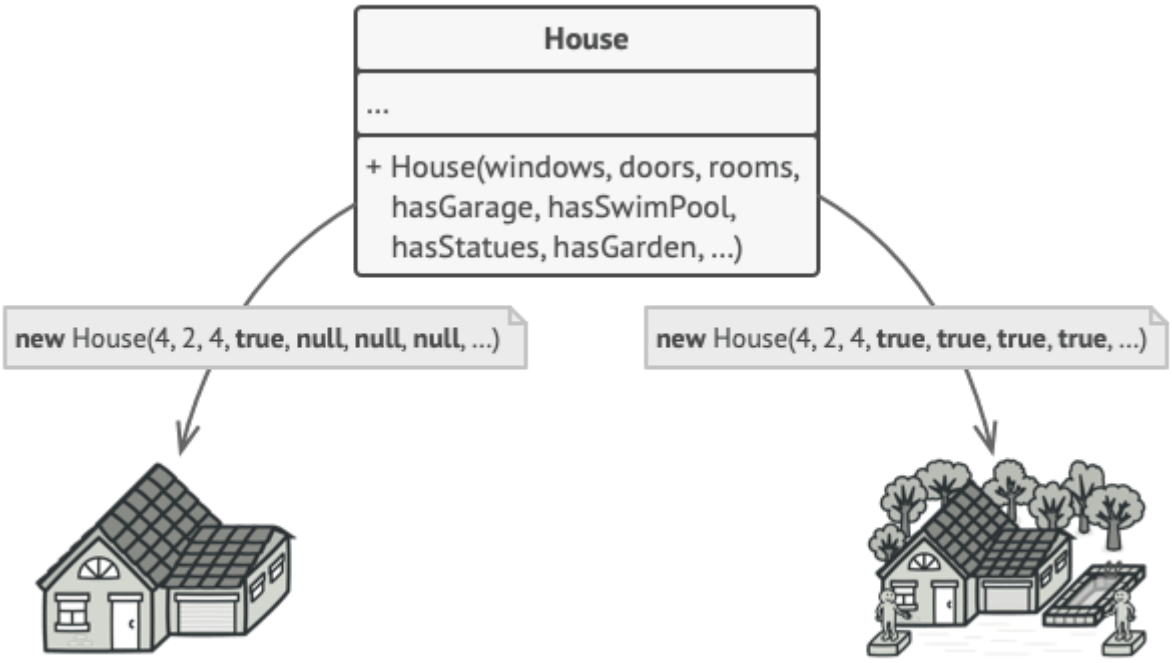



Sie könnten das Programm zu komplex machen, wenn Sie für jede mögliche Konfiguration eines Objekts eine Unterklasse erstellen.

Betrachten wir beispielsweise die Erstellung eines `House`Objekts. Um ein einfaches Haus zu bauen, müssen vier Wände und ein Boden errichtet, eine Tür eingebaut, zwei Fenster angebracht und ein Dach gebaut werden. Was aber, wenn Sie ein größeres, helleres Haus mit Garten und weiteren Annehmlichkeiten (wie Heizung, Sanitäranlagen und Elektroinstallationen) wünschen?

Die einfachste Lösung besteht darin, die Basisklasse zu erweitern `House`und eine Reihe von Unterklassen zu erstellen, um alle Parameterkombinationen abzudecken. Allerdings werden Sie am Ende eine beträchtliche Anzahl von Unterklassen haben. Jeder neue Parameter, wie z. B. der Veranda-Stil, erfordert eine weitere Erweiterung dieser Hierarchie.

Es gibt einen anderen Ansatz, der ohne die Bildung von Unterklassen auskommt. Sie können direkt in der Basisklasse einen riesigen Konstruktor `House`mit allen möglichen Parametern erstellen, die das Hausobjekt steuern. Dieser Ansatz macht zwar Unterklassen überflüssig, schafft aber ein anderes Problem.

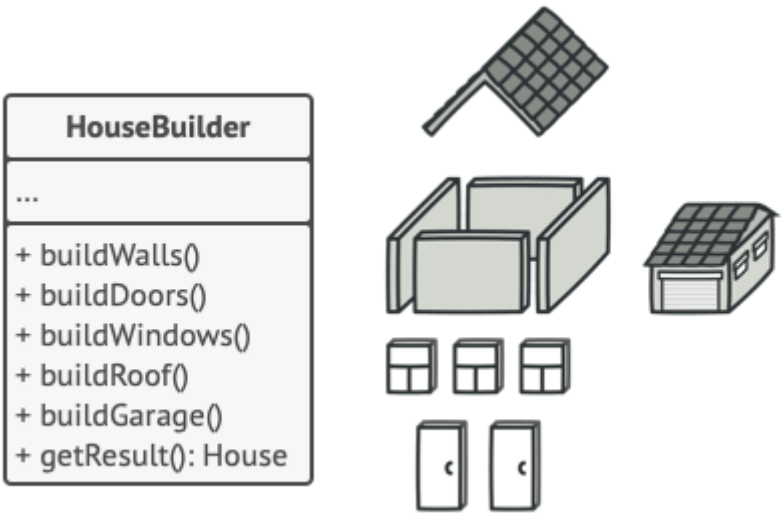



Der Konstruktor mit vielen Parametern hat seinen Nachteil: Nicht alle Parameter werden immer benötigt.

In den meisten Fällen werden die meisten Parameter ungenutzt bleiben, was **die Konstruktoraufrufe ziemlich hässlich** macht . Beispielsweise haben nur wenige Häuser Swimmingpools, sodass die Parameter für Swimmingpools in neun von zehn Fällen nutzlos sind.

## Lösung

Das Builder-Muster schlägt vor, dass Sie den Objektkonstruktionscode aus seiner eigenen Klasse extrahieren und ihn in separate Objekte, sogenannte *Builder* , verschieben .

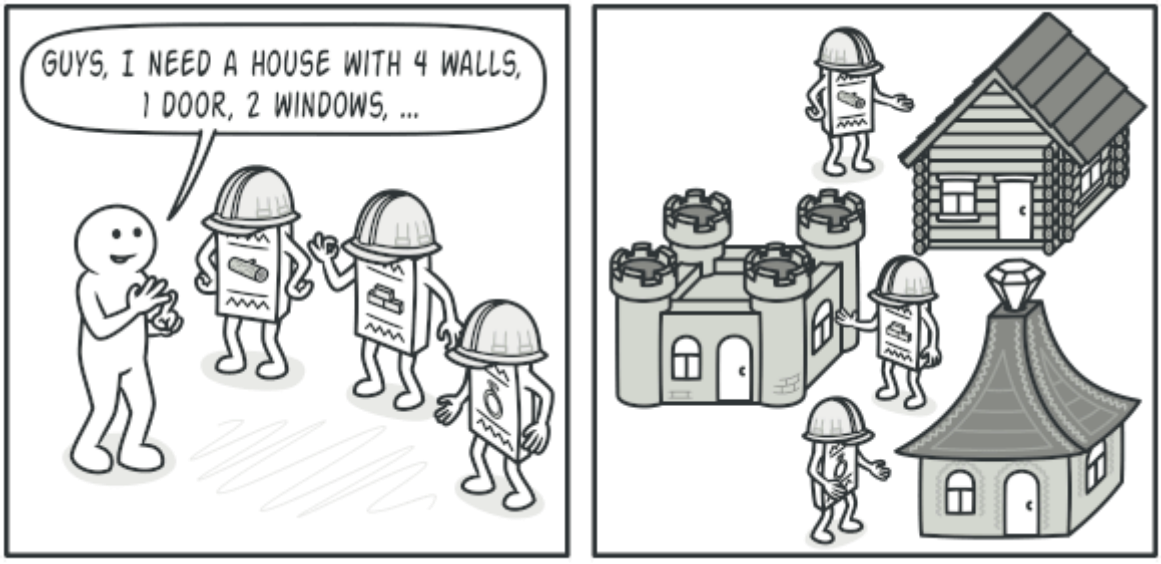



Mit dem Builder-Muster können Sie komplexe Objekte schrittweise erstellen. Während der Erstellung lässt der Builder keinen Zugriff anderer Objekte auf das Produkt zu.

Das Muster organisiert die Objekterstellung in einer Reihe von Schritten ( `buildWalls`, `buildDoor`, usw.). Um ein Objekt zu erstellen, führen Sie eine Reihe dieser Schritte auf einem Builder-Objekt aus. Wichtig ist, dass Sie nicht alle Schritte aufrufen müssen. Sie können nur die Schritte aufrufen, die für die Erstellung einer bestimmten Objektkonfiguration erforderlich sind.

Einige der Konstruktionsschritte erfordern möglicherweise eine unterschiedliche Implementierung, wenn Sie verschiedene Darstellungen des Produkts erstellen müssen. Beispielsweise können die Wände einer Hütte aus Holz gebaut werden, die Mauern einer Burg müssen jedoch aus Stein gebaut werden.

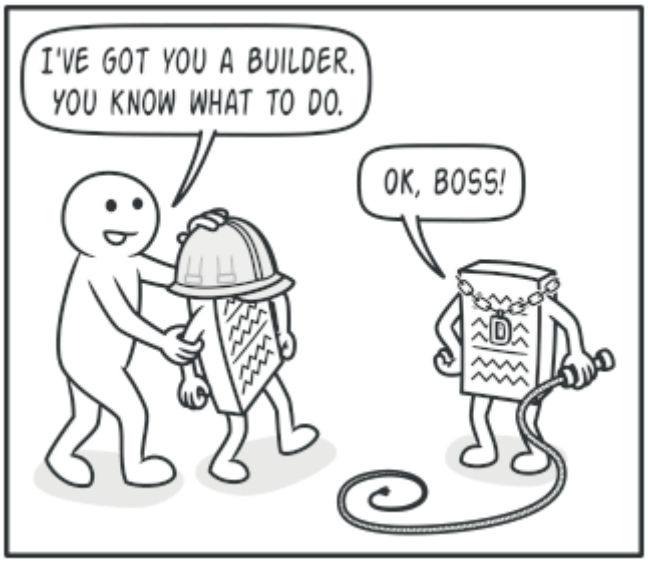
In diesem Fall können Sie mehrere unterschiedliche Builder-Klassen erstellen, die dieselben Erstellungsschritte auf unterschiedliche Weise implementieren. Anschließend können Sie diese Builder im Erstellungsprozess (d. h. in einer geordneten Reihe von Aufrufen der Erstellungsschritte) verwenden, um unterschiedliche Arten von Objekten zu erzeugen.




Verschiedene Builder führen dieselbe Aufgabe auf unterschiedliche Weise aus. Stellen Sie sich beispielsweise einen Baumeister vor, der alles aus Holz und Glas baut, einen zweiten, der alles aus Stein und Eisen baut, und einen dritten, der Gold und Diamanten verwendet. Durch Aufruf derselben Schritte erhalten Sie vom ersten Baumeister ein normales Haus, vom zweiten ein kleines Schloss und vom dritten einen Palast. Dies funktioniert jedoch nur, wenn der Client-Code, der die Bauschritte aufruft, über eine gemeinsame Schnittstelle mit den Baumeistern interagieren kann.

*Direktor*

Sie können noch einen Schritt weiter gehen und eine Reihe von Aufrufen der Builder-Schritte, die Sie zum Erstellen eines Produkts verwenden, in eine separate Klasse namens *„director“* extrahieren . Die Director-Klasse definiert die Reihenfolge, in der die Erstellungsschritte ausgeführt werden, während der Builder die Implementierung dieser Schritte bereitstellt.

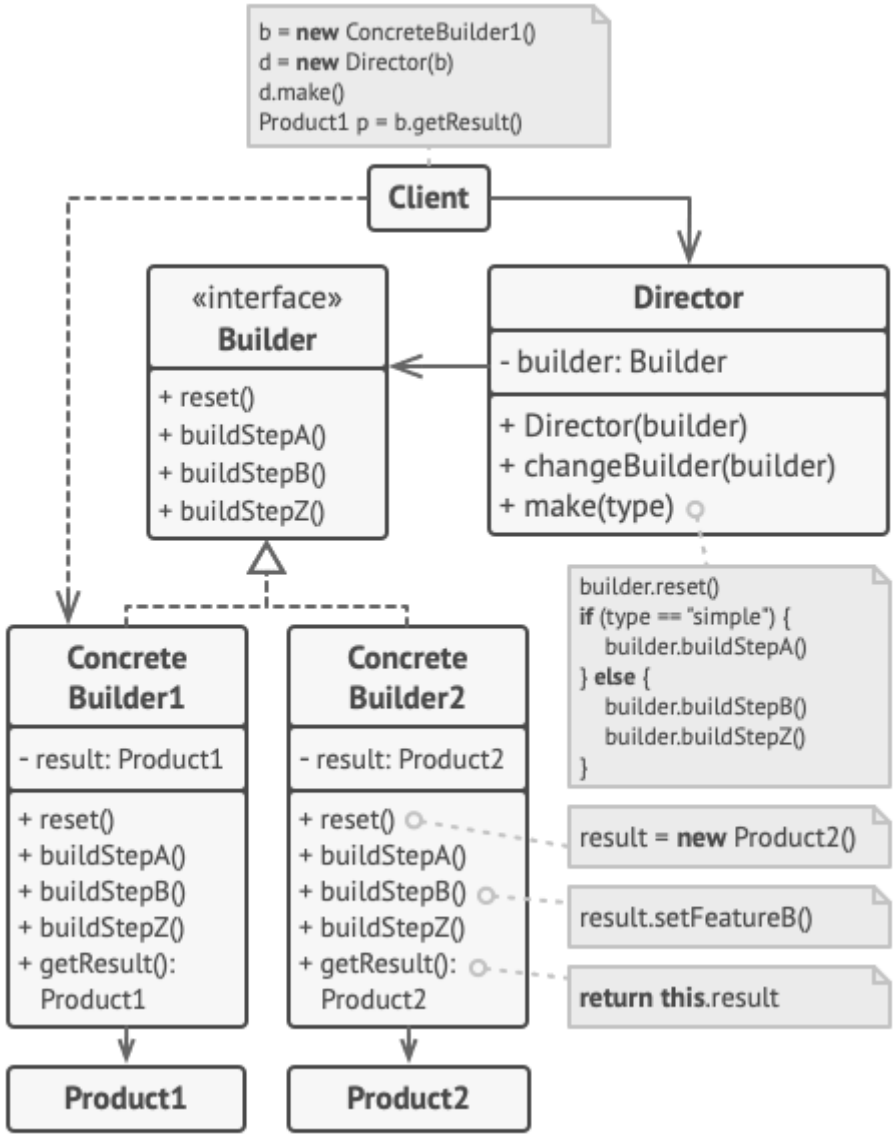



Der Regisseur weiß, welche Bauschritte ausgeführt werden müssen, um ein funktionierendes Produkt zu erhalten.

Eine Director-Klasse in Ihrem Programm ist nicht unbedingt erforderlich. Sie können die Erstellungsschritte jederzeit in einer bestimmten Reihenfolge direkt aus dem Client-Code aufrufen. Die Director-Klasse eignet sich jedoch gut für verschiedene Konstruktionsroutinen, die Sie im gesamten Programm wiederverwenden können.

Darüber hinaus verbirgt die Director-Klasse die Details der Produktkonstruktion vollständig vor dem Client-Code. Der Client muss lediglich einen Builder mit einem Director verknüpfen, die Konstruktion mit dem Director starten und das Ergebnis vom Builder abrufen.

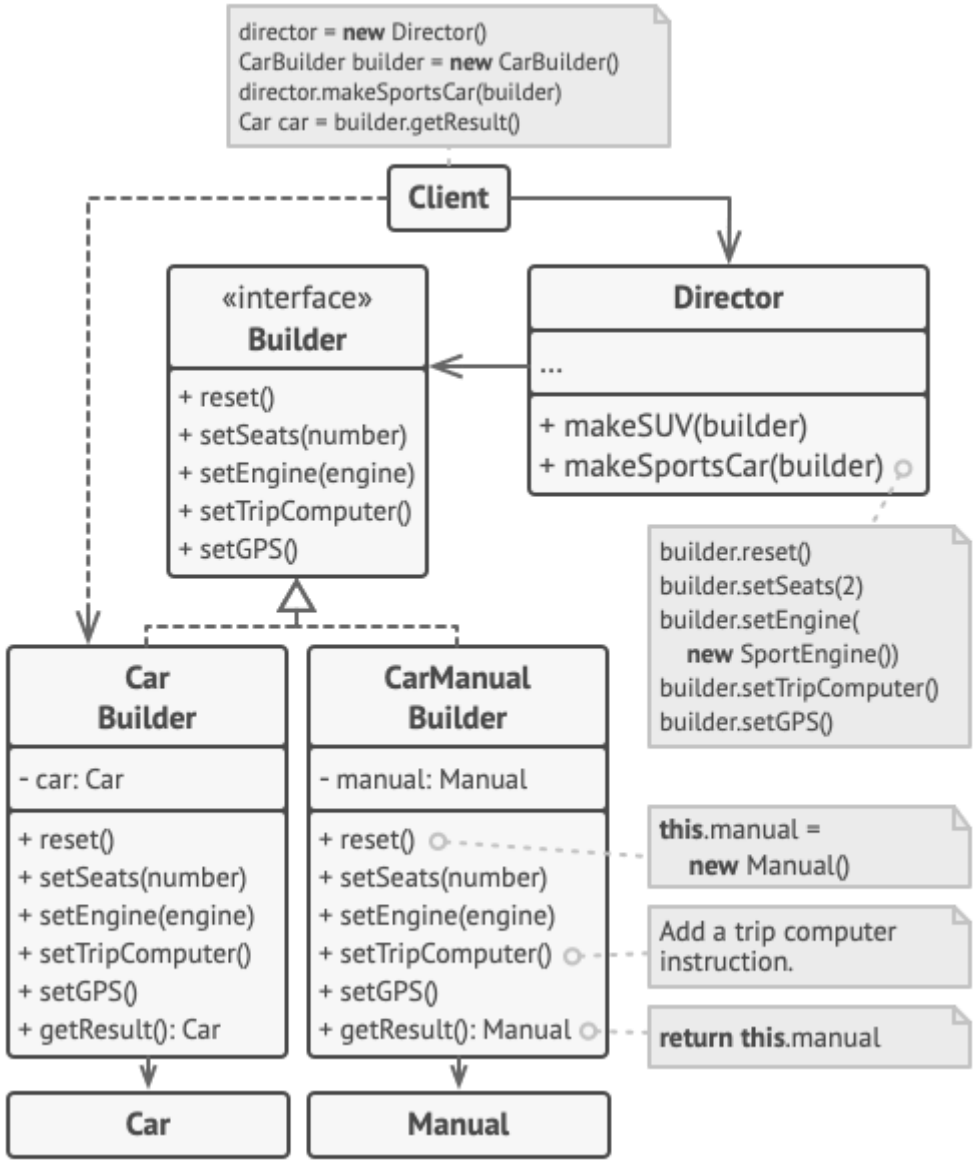
Struktur




1. Die **Builder-** Schnittstelle deklariert Produktkonstruktionsschritte, die allen Builder-Typen gemeinsam sind.
2. **Concrete Builders** bieten unterschiedliche Implementierungen der Konstruktionsschritte an. Concrete Builders stellen möglicherweise Produkte her, die nicht der gemeinsamen Schnittstelle folgen.
3. **Produkte** sind resultierende Objekte. Von verschiedenen Buildern erstellte Produkte müssen nicht derselben Klassenhierarchie oder Schnittstelle angehören.
4. Die **Director-** Klasse definiert die Reihenfolge, in der Konstruktionsschritte aufgerufen werden, sodass Sie bestimmte Produktkonfigurationen erstellen und wiederverwenden können.
5. Der **Client** muss eines der Builder-Objekte mit dem Director verknüpfen. Normalerweise geschieht dies nur einmal über Parameter des Konstruktors des Directors. Anschließend verwendet der Director dieses Builder-Objekt für alle weiteren Konstruktionen. Es gibt jedoch einen alternativen Ansatz, wenn der Client das Builder-Objekt an die Produktionsmethode des Directors übergibt. In diesem Fall können Sie bei jeder Produktion mit dem Director einen anderen Builder verwenden.

## Pseudocode

Dieses Beispiel des **Builder-** Musters veranschaulicht, wie Sie beim Erstellen unterschiedlicher Produkttypen, z. B. Autos, denselben Objektkonstruktionscode wiederverwenden und die entsprechenden Handbücher dafür erstellen können.




Das Beispiel der schrittweisen Konstruktion von Autos und der zu den jeweiligen Automodellen passenden Benutzerhandbücher.

Ein Auto ist ein komplexes Objekt, das auf hundert verschiedene Arten konstruiert werden kann. Anstatt die `Car`Klasse mit einem riesigen Konstruktor aufzublähen, haben wir den Code für die Automontage in eine separate Auto-Builder-Klasse extrahiert. Diese Klasse verfügt über eine Reihe von Methoden zur Konfiguration verschiedener Autoteile.

Wenn der Clientcode ein spezielles, fein abgestimmtes Automodell zusammenbauen muss, kann er direkt mit dem Builder zusammenarbeiten. Andererseits kann der Client

die Montage an die Director-Klasse delegieren, die mit einem Builder mehrere der beliebtesten Automodelle konstruieren kann.

Es mag Sie schockieren, aber jedes Auto benötigt ein Handbuch (im Ernst, wer liest die schon?). Das Handbuch beschreibt jede Funktion des Autos, daher variieren die Details in den Handbüchern je nach Modell. Deshalb ist es sinnvoll, einen vorhandenen Konstruktionsprozess sowohl für echte Autos als auch für die jeweiligen Handbücher wiederzuverwenden. Natürlich ist das Erstellen eines Handbuchs nicht dasselbe wie das Bauen eines Autos, und deshalb müssen wir eine weitere Builder-Klasse bereitstellen, die auf das Erstellen von Handbüchern spezialisiert ist. Diese Klasse implementiert dieselben Konstruktionsmethoden wie ihr Geschwistermodell zum Autobau, aber anstatt Autoteile herzustellen, beschreibt sie diese. Indem wir diese Builder an dasselbe Director-Objekt übergeben, können wir entweder ein Auto oder ein Handbuch konstruieren.

Der letzte Teil ist das Abrufen des resultierenden Objekts. Ein Metallauto und eine gedruckte Anleitung sind zwar verwandt, aber dennoch sehr unterschiedliche Dinge. Wir können keine Methode zum Abrufen von Ergebnissen im Director platzieren, ohne den Director an konkrete Produktklassen zu koppeln. Daher erhalten wir das Ergebnis der Konstruktion vom Builder, der die Aufgabe ausgeführt hat.

```
// Using the Builder pattern makes sense only when your products

// are quite complex and require extensive configuration. The
// following two products are related, although they don't have
// a common interface.
class Car is
    // A car can have a GPS, trip computer and some number of
    // seats. Different models of cars (sports car, SUV,
    // cabriolet) might have different features installed or
    // enabled.

class Manual is
    // Each car should have a user manual that corresponds to
    // the car's configuration and describes all its features.


// The builder interface specifies methods for creating the
// different parts of the product objects.
interface Builder is
    method reset()
    method setSeats(...)
    method setEngine(...)
    method setTripComputer(...)
    method setGPS(...)

// The concrete builder classes follow the builder interface and
// provide specific implementations of the building steps. Your
// program may have several variations of builders, each
// implemented differently.
class CarBuilder implements Builder is
    private field car:Car

    // A fresh builder instance should contain a blank product
    // object which it uses in further assembly.
    constructor CarBuilder() is
        this.reset()

    // The reset method clears the object being built.
    method reset() is
        this.car = new Car()

    // All production steps work with the same product instance.
    method setSeats(...) is
        // Set the number of seats in the car.

    method setEngine(...) is
        // Install a given engine.

    method setTripComputer(...) is
        // Install a trip computer.

    method setGPS(...) is
        // Install a global positioning system.
```

```
    // Concrete builders are supposed to provide their own
    // methods for retrieving results. That's because various
    // types of builders may create entirely different products
    // that don't all follow the same interface. Therefore such
    // methods can't be declared in the builder interface (at
    // least not in a statically-typed programming language).
    //
    // Usually, after returning the end result to the client, a
    // builder instance is expected to be ready to start
    // producing another product. That's why it's a usual
    // practice to call the reset method at the end of the
    // `getProduct` method body. However, this behavior isn't
    // mandatory, and you can make your builder wait for an
    // explicit reset call from the client code before disposing
    // of the previous result.
    method getProduct():Car is
        product = this.car
        this.reset()
        return product

// Unlike other creational patterns, builder lets you construct
// products that don't follow the common interface.
class CarManualBuilder implements Builder is
    private field manual:Manual

    constructor CarManualBuilder() is
        this.reset()

    method reset() is
        this.manual = new Manual()

    method setSeats(...) is
        // Document car seat features.

    method setEngine(...) is
        // Add engine instructions.

    method setTripComputer(...) is
        // Add trip computer instructions.

    method setGPS(...) is
        // Add GPS instructions.

    method getProduct():Manual is
        // Return the manual and reset the builder.
```

```
// The director is only responsible for executing the building
// steps in a particular sequence. It's helpful when producing
// products according to a specific order or configuration.
// Strictly speaking, the director class is optional, since the
// client can control builders directly.
class Director is
    // The director works with any builder instance that the
    // client code passes to it. This way, the client code may
    // alter the final type of the newly assembled product.
    // The director can construct several product variations
    // using the same building steps.
    method constructSportsCar(builder: Builder) is
        builder.reset()
        builder.setSeats(2)
        builder.setEngine(new SportEngine())
        builder.setTripComputer(true)
        builder.setGPS(true)

    method constructSUV(builder: Builder) is
        // ...


// The client code creates a builder object, passes it to the
// director and then initiates the construction process. The end
// result is retrieved from the builder object.
class Application is

    method makeCar() is
        director = new Director()

        CarBuilder builder = new CarBuilder()
        director.constructSportsCar(builder)
        Car car = builder.getProduct()

        CarManualBuilder builder = new CarManualBuilder()
        director.constructSportsCar(builder)

        // The final product is often retrieved from a builder
        // object since the director isn't aware of and not
        // dependent on concrete builders and products.
        Manual manual = builder.getProduct()
```